

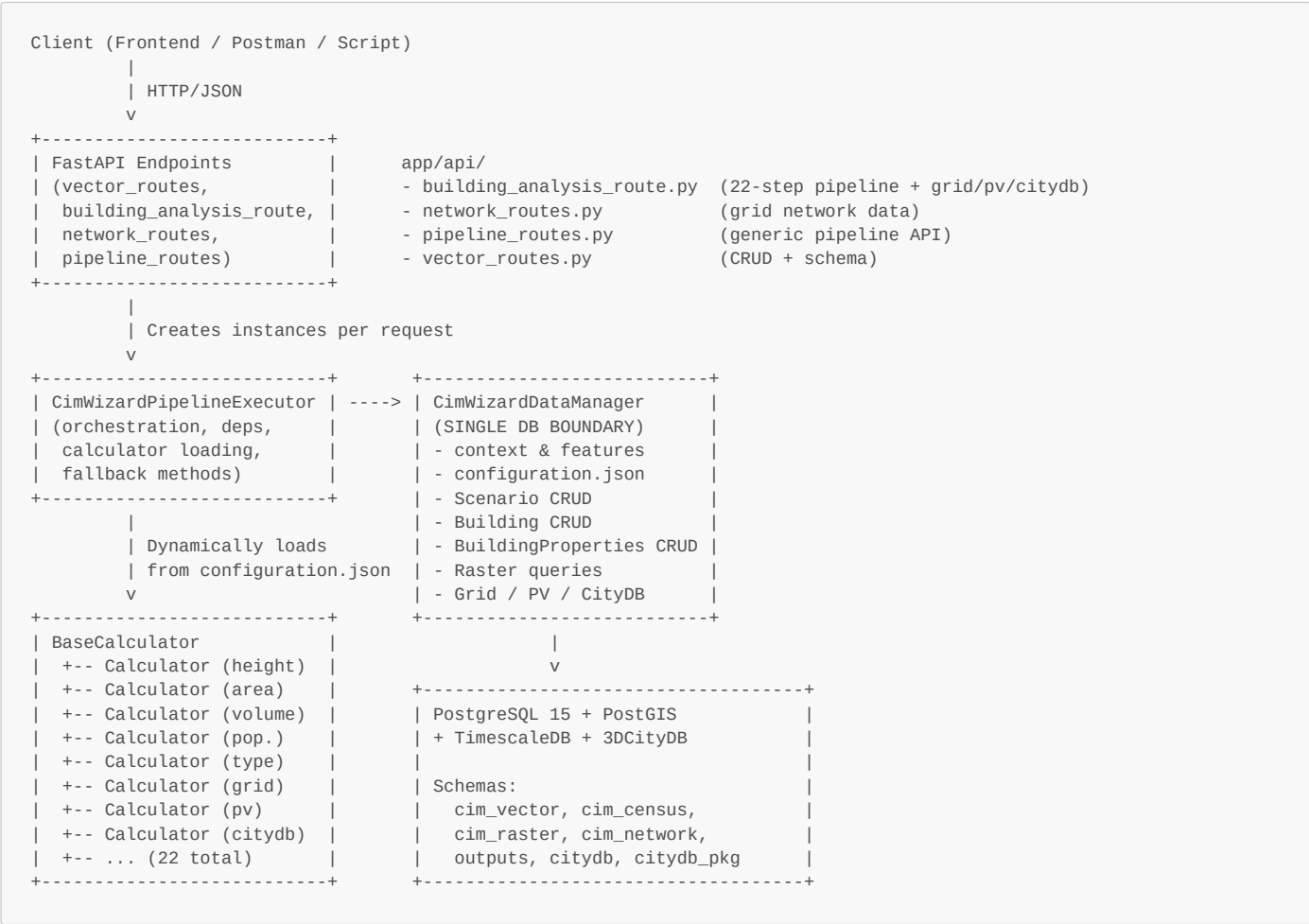
Date: 2026-03-10
Scope: Architecture, calculator pipeline, API compliance, deployment

[illegible]

1. System Overview

The CIM Wizard system consists of three interconnected projects:

Project	Role	Technology	Port
cim-database	Dockerized PostgreSQL/PostGIS database	PostgreSQL 15 + PostGIS + TimescaleDB + 3DCityDB	15432
cim_wizard_integrated_2026	Backend API server	Python / FastAPI / SQLAlchemy	8000
coesi-frontend-main_2026	Frontend web application	React / TypeScript / Vite	5173



Key architectural rule: only CimWizardDataManager touches the database. Routes and calculators delegate all persistence to DataManager methods.

2. Backend Architecture

2.1 Core OOP Pattern

The backend enforces a strict separation of concerns across four layers:



- CimWizardDataManager** -- The single data-access boundary for the entire application. Every database read, write, and delete goes through this class. It also manages context (project/scenario IDs), feature storage, configuration, service access, and field normalization. Routes and calculators never call `db.query()` or `db.commit()` directly.
- CimWizardPipelineExecutor** -- Pure orchestration. It reads `configuration.json` to know which calculator class to load for each feature, resolves dependencies between features, selects the right method (with fallback), and stores results back through the DataManager.

3. **BaseCalculator / Calculators** -- Domain logic only. Each calculator inherits from BaseCalculator, which provides convenience wrappers for logging, validation, feature access, and a `save_property_batch()` helper that delegates to DataManager. Calculators never import SQLAlchemy models or execute raw SQL.
4. **Endpoint routes** -- Thin request/response handlers. Each route creates a DataManager (bound to the current request session), a PipelineExecutor, and orchestrates a calculation chain. All database interaction in the route goes through DataManager methods.

```
Endpoint function
|
|-- Creates DataManager(db_session)
|-- Creates PipelineExecutor(data_manager)
|-- Sets context (project_id, scenario_id, boundary)
|-- Defines calculation_chain = [{feature, method}, ...]
|
|-- For each step in chain:
|   executor.execute_feature(feature_name, method_name)
|   |
|   |-- Loads calculator class from configuration.json
|   |-- Checks dependencies via has_feature() / get_context()
|   |-- Calls calculator.method()
|   |-- Calculator stores result via data_manager.set_feature()
|   |-- Calculator persists to DB via data_manager.upsert_*(*)
|
|-- Returns JSON response (no separate DB-save loop needed)
```

2.2 CimWizardDataManager

File: `app/core/data_manager.py`

Responsibilities:

- **Context storage:** `project_id`, `scenario_id`, service references
- **Feature storage:** `calculated_features` dict holds all computed values
- **Configuration:** Loads and caches `configuration.json` at init
- **Feature access:** `set_feature()`, `get_feature()`, `has_feature()`
- **Config access:** `get_feature_config(name)`, `get_pipeline_config(name)`
- **Scenario CRUD:** `save_scenario()`, `get_scenario()`, `create_scenario_from_baseline()`, `delete_project_data()`
- **Building CRUD:** `save_building()`, `get_building()`, `get_buildings_geojson()`, spatial queries
- **BuildingProperties CRUD:** `upsert_building_properties_batch()`, `upsert_building_property_fields()`, `get_building_properties()`
- **Raster queries:** `check_raster_table()`, `query_raster_value()`
- **Census boundary:** `update_census_boundary()`

Key methods:

Method	Purpose
<code>set_context(**kwargs)</code>	Set <code>project_id</code> , <code>scenario_id</code> , etc.
<code>set_feature(name, value)</code>	Store a calculated Feature result
<code>get_feature(name)</code>	Retrieve a calculated feature
<code>has_feature(name)</code>	Check if a feature has been calculated
<code>get_feature_config(name)</code>	Get calculator config for a feature from <code>configuration.json</code>
<code>save_scenario(...)</code>	Upsert a ProjectScenario from GeoJSON geometry
<code>save_building(...)</code>	Upsert a Building row from a GeoJSON Feature
<code>upsert_building_properties_batch(...)</code>	Bulk upsert a single property column across many buildings
<code>upsert_building_property_fields(...)</code>	Upsert arbitrary fields on a single BuildingProperties row
<code>get_buildings_geojson(...)</code>	Return GeoJSON FeatureCollection with baseline/delta merge
<code>delete_project_data(...)</code>	Cascading delete (building, scenario, or project)
<code>check_raster_table(name)</code>	Check if a raster table exists and has data
<code>query_raster_value(table, lon, lat)</code>	Get raster cell value at a point

2.3 CimWizardPipelineExecutor

File: `app/core/pipeline_executor.py`

Responsibilities:

- **Calculator loading:** Dynamically imports calculator classes using `class_path` and `class_name` from `configuration.json`. Caches instances in `calculator_cache`.

- **Dependency resolution:** `check_dependencies()` verifies all inputs are available before running a calculator. `_topological_sort()` can order features by dependency graph.
- **Method selection with fallback:** When `execute_feature()` is called without an explicit method, it tries each method defined in `configuration.json` in priority order until one succeeds.
- **Pipeline execution:** `execute_pipeline()`, `execute_predefined_pipeline()`, `execute_explicit_pipeline()` for batch execution.

Key methods:

Method	Purpose
<code>execute_feature(name, method)</code>	Run one calculator. If method is specified, only try that. Otherwise try methods in priority order.
<code>check_dependencies(deps)</code>	Returns True if all dependency features are available
<code>get_calculator_instance(name)</code>	Load and cache a calculator class from <code>configuration.json</code>
<code>execute_pipeline(features)</code>	Run multiple features with automatic dependency resolution
<code>execute_predefined_pipeline(name)</code>	Run a named pipeline from <code>configuration.json</code> <code>predefined_pipelines</code>

Fallback mechanism: In `configuration.json`, each feature can define multiple methods with priorities:

```
"building_population": {
  "methods": [
    {"priority": 1, "method_name": "calculate_from_volume_distribution",
      "input_dependencies": ["building_volume", "census_population"]},
    {"priority": 2, "method_name": "by_census_osm",
      "input_dependencies": ["scenario_census_boundary", "building_type", "building_volume"]}
  ]
}
```

If priority-1 method fails (e.g., missing `census_population`), the executor falls back to priority-2. This only works when `execute_feature()` is called without an explicit method name.

2.4 BaseCalculator and Calculators

Directory: `app/calculators/`

Every calculator inherits from `BaseCalculator` (`app/calculators/base_calculator.py`), which provides:

- Pipeline executor and data manager references
- Logging helpers: `log_info()`, `log_error()`, `log_warning()`, `log_success()`, `log_failure()`
- Feature helpers: `get_feature()`, `set_feature()`
- Validation helpers: `validate_input()`, `validate_dict()`, `validate_geometry()`, `validate_numeric()`
- Properties: `project_id`, `scenario_id`
- DB persistence: `save_property_batch()` -- delegates to `DataManager`

Each calculator:

1. Inherits from `BaseCalculator` and calls `super().__init__(pipeline_executor)`
2. Reads dependencies via `self.get_feature(name)` or `self.pipeline.get_feature_safely(name)`
3. Performs domain-specific computation
4. Stores result via `self.set_feature(name, value)`
5. Persists to DB via `self.data_manager.upsert_building_properties_batch(...)` or `self.save_property_batch(...)`
6. Returns the result (or None on failure)

Calculators never import SQLAlchemy models or execute raw queries. All database interaction is delegated to `DataManager`.

Example:

```
from app.calculators.base_calculator import BaseCalculator

class BuildingVolumeCalculator(BaseCalculator):
    def __init__(self, pipeline_executor):
        super().__init__(pipeline_executor)

    def calculate_from_height_and_area(self):
        heights = self.get_feature('building_height')
        areas = self.get_feature('building_area')
        # ... compute volumes ...
        self.set_feature('building_volume', result)
        self.save_property_batch(buildings, 'volume', volumes)
        return result
```

2.5 configuration.json

File: `app/core/configuration.json`

This file is the registry for all calculators. It defines:

- **features:** Each feature's calculator class, methods, dependencies, and constraints
- **predefined_pipelines:** Named sequences of features for common tasks
- **global_settings:** Timeouts, retries, caching, validation flags

The PipelineExecutor reads this at runtime to know how to load calculators and what dependencies to check. Adding a new calculator means adding an entry here and placing the Python class in `app/calculators/`.

2.6 Endpoint Routes

Three route files define different pipeline entry points:

Route file	Endpoint	Steps	Purpose
<code>building_analysis_route.py</code>	POST <code>/api/v1/building/execute_building_analysis</code>	16	Full pipeline: physical properties + demographics + classification + LoD 1.2. Used by frontend.
<code>complete_chain_route.py</code>	POST <code>/api/v1/complete/execute_complete_chain</code>	16	Deprecated. Older implementation with known bugs (non-UUID IDs, no DB save). Use <code>building_analysis</code> instead.
<code>pipeline_routes.py</code>	POST <code>/api/v1/pipeline/execute</code>	Dynamic	Generic pipeline executor. Client specifies which features to run.

Each route creates its own DataManager + PipelineExecutor and defines its own calculation chain. The chain is a list of `{feature_name, method_name}` dicts iterated sequentially.

3. Calculator Reference

3.1 Full Calculator Inventory

There are **22 calculators** registered in `configuration.json`. Each one targets specific columns in the database:

#	Feature Name	Calculator Class	Method(s)	DB Column(s) Populated
1	<code>scenario_geo</code>	ScenarioGeoCalculator	<code>calculate_from_scenario_geo</code> , <code>calculate_from_buildings_geo</code>	<code>project_boundary</code> , <code>project_center</code> on <code>cim_wizard_project_scenario</code>
2	<code>scenario_census_boundary</code>	ScenarioCensusBoundaryCalculator	<code>calculate_from_census_api</code>	<code>census_boundary</code> on <code>cim_wizard_project_scenario</code>
3	<code>building_geo</code>	BuildingGeoCalculator	<code>calculate_from_scenario_census_geo</code> , <code>calculate_from_building_geo</code>	Rows in <code>cim_wizard_building</code> (<code>building_id</code> , <code>geometry</code> , <code>source</code>)
4	<code>building_props</code>	BuildingPropsCalculator	<code>init</code>	Initializes rows in <code>cim_wizard_building_properties</code> (skeleton records)
5	<code>building_height</code>	BuildingHeightCalculator	<code>calculate_from_raster_tiles</code>	<code>height</code> on <code>cim_wizard_building_properties</code>
6	<code>building_area</code>	BuildingAreaCalculator	<code>calculate_from_geometry</code>	<code>area</code> on <code>cim_wizard_building_properties</code>
7	<code>filter_res</code>	BuildingResidentialFilterCalculator	<code>calculate_filter_res</code>	<code>filter_res</code> on <code>cim_wizard_building_properties</code>
8	<code>building_volume</code>	BuildingVolumeCalculator	<code>calculate_from_height_and_area</code>	<code>volume</code> on <code>cim_wizard_building_properties</code>
9	<code>building_n_floors</code>	BuildingNFloorsCalculator	<code>estimate_by_height</code>	<code>number_of_floors</code> on <code>cim_wizard_building_properties</code>
10	<code>census_population</code>	CensusPopulationCalculator	<code>calculate_from_census_boundary</code>	(<i>intermediate -- total population from census sections</i>)
11	<code>building_type</code>	BuildingTypeCalculator	<code>by_census_osm</code>	<code>type</code> on <code>cim_wizard_building_properties</code> (TABULA classification)
12	<code>building_population</code>	BuildingPopulationCalculator	<code>calculate_from_volume_distribution</code> , <code>by_census_osm</code>	<code>n_people</code> on <code>cim_wizard_building_properties</code>
13	<code>building_n_families</code>	BuildingNFamiliesCalculator	<code>calculate_from_population</code> , <code>by_census_osm</code>	<code>n_family</code> on <code>cim_wizard_building_properties</code>

#	Feature Name	Calculator Class	Method(s)	DB Column(s) Populated
14	building_construction_year	BuildingConstructionYearCalculator	by_census_osm	const_year, const_period_census, const_tabula on cim_wizard_building_properties
15	building_demographic	BuildingDemographicCalculator	by_census_osm	Orchestrates n_people + n_family from census integration
16	building_geo_lod12	BuildingGeoLod12Calculator	by_footprint_height	building_surfaces_lod12 on cim_wizard_building (3D geometry)
17	envelope_efficiency	EnvelopeEfficiencyCalculator	assign_random	envelope_efficiency on cim_wizard_building_properties
18	fmu_assign	FmuAssignCalculator	frassinetto	fmu_file on cim_wizard_building_properties
19	building_z_value	BuildingZValueCalculator	calculate_from_dtm	z_value on cim_wizard_building_properties
20	building_name	BuildingNameCalculator	assign_sequential	building_name on cim_wizard_building (BUI-0001 format)
21	grid_generator	GridGeneratorCalculator	assign_grid	grid_id on cim_wizard_project_scenario
22	pv_generator	PvGeneratorCalculator	assign_pv_to_buildings	pv_ids on cim_wizard_building_properties

Additionally, `CitydbMapperCalculator` provides `map_scenario_to_citydb` (writes to `citydb` schema) and `generate_cityjson` (reads from `cim_vector` and returns CityJSON v1.1).

3.2 Building Analysis Pipeline (22 steps)

Endpoint: `POST /api/v1/building/execute_building_analysis`

This is the main pipeline used by the frontend and Postman. It runs 22 calculators sequentially and populates every DB column:

Step	Feature	Method	DB Column / Target
1	scenario_geo	calculate_from_scenario_geo	project_boundary, project_center
2	scenario_census_boundary	calculate_from_census_api	census_boundary
3	building_geo	calculate_from_scenario_census_geo	cim_wizard_building rows
4	building_props	init	building_properties (skeleton)
5	building_height	calculate_from_raster_tiles	height
6	building_area	calculate_from_geometry	area
7	filter_res	calculate_filter_res	filter_res (residential filter)
8	building_volume	calculate_from_height_and_area	volume
9	building_n_floors	estimate_by_height	number_of_floors
10	census_population	calculate_from_census_boundary	(intermediate: total pop)
11	building_type	by_census_osm	type (TABULA classification)
12	building_population	calculate_from_volume_distribution	n_people
13	building_n_families	calculate_from_population	n_family
14	building_construction_year	by_census_osm	const_year, const_period_census, const_tabula
15	building_demographic	by_census_osm	(orchestrates n_people + n_family)
16	building_geo_lod12	by_footprint_height	building_surfaces_lod12
17	envelope_efficiency	assign_random	envelope_efficiency
18	fmu_assign	frassinetto	fmu_file
19	building_z_value	calculate_from_dtm	z_value (avg DTM under footprint)
20	building_name	assign_sequential	building_name (BUI-0001, ...)

Steps 21 (grid_generator) and 22 (pv_generator) are not part of the automatic pipeline; they are triggered via their own POST endpoints.

3.3 Legacy Complete Chain (deprecated)

Endpoint: `POST /api/v1/complete/execute_complete_chain`

This is an older implementation in `complete_chain_route.py`. It has known issues (non-UUID IDs, wrong method names, no DB saving) and should not be used. Use the building analysis endpoint above instead.

4. Database Schema

The database image (`cim-database/Dockerfile`) is built as a multi-stage Docker image combining:

- **PostgreSQL 15** with **PostGIS 3.4** (base spatial database)

- **TimescaleDB** (time-series hypertables in the **outputs** schema)
- **3DCityDB v4** (CityGML schema in the **citydb** / **citydb_pkg** schemas)
- **Energy ADE** and **Utility Network ADE** (3DCityDB extensions)

4.1 CIM Wizard schemas (as-is)

Current database schemas as implemented. See [paper/generate_data_list_xlsx.py](#) and [paper/data-list.md](#) for the target ECDT data inventory.

cim_vector, cim_census, cim_raster, cim_network

```
+-----+
| cim_vector.cim_wizard_project_scenario |
+-----+
| PK project_id    VARCHAR(100) |
| PK scenario_id  VARCHAR(100) |
| project_name    VARCHAR(255) |
| scenario_name   VARCHAR(255) |
| project_boundary GEOMETRY(POLYGON,4326) |
| project_center  GEOMETRY(POINT,4326) |
| project_zoom    INTEGER [default: 15] |
| project_crs     INTEGER [default: 4326] |
| census_boundary GEOMETRY(MULTIPOLY,4326) |
| FK grid_id      UUID --> cim_network.network_scenarios.grid_id |
| created_at      TIMESTAMP WITH TZ |
| updated_at      TIMESTAMP WITH TZ |
+-----+
| project_id + scenario_id (logical FK) |
v
+-----+
| cim_vector.cim_wizard_building_properties |
+-----+
| PK scenario_id  VARCHAR(100) |
| PK building_id  VARCHAR(100) -----+
| PK lod          INTEGER [default: 0] |
| project_id      VARCHAR(100) |
| height          FLOAT |
| area            FLOAT |
| volume          FLOAT |
| number_of_floors FLOAT |
| type            VARCHAR(50) |
| envelope_efficiency VARCHAR(20) |
| fmu_file        VARCHAR(255) |
| const_period_census VARCHAR(10) |
| const_year      INTEGER |
| const_tabula    VARCHAR(15) |
| n_people        INTEGER |
| n_family        INTEGER |
| created_at      TIMESTAMP WITH TZ |
| updated_at      TIMESTAMP WITH TZ |
+-----+
| building_id (logical FK) |
v
+-----+
| cim_vector.cim_wizard_building |
+-----+
| PK building_id  VARCHAR(100) |
| lod            INTEGER [default: 0] |
| building_geometry GEOMETRY(GEOM,4326) |
| building_geometry_source VARCHAR(50) |
| census_id       BIGINT |
| z_value         DOUBLE PRECISION |
| building_name   VARCHAR(100) |
| pv_ids          UUID[] (GIN indexed) |
| building_surfaces_lod12 JSON |
| created_at      TIMESTAMP WITH TZ |
| updated_at      TIMESTAMP WITH TZ |
+-----+
+-----+
| cim_vector.pv |
+-----+
| PK pv_id       UUID |
| FK building_id UUID --> cim_wizard_building.building_id |
| fid           BIGINT |
| slope         FLOAT |
| num           FLOAT |
| area_reale    FLOAT |
| number        INTEGER |
| s             FLOAT |
```

```
|      index_righ    BIGINT      |
|      id_pod        FLOAT      |
|      pv_geometry   GEOMETRY(MULTIPOLY,4326) |
+-----+

+-----+
|      cim_census.censusgeo      |
+-----+
| PK id      SERIAL      |
| UQ SEZ2011 BIGINT      |
|      geometry GEOMETRY(MULTIPOLYGON,4326) |
|      P1..P66  (population columns)      |
|      E1..E31  (building age columns)     |
|      ST1..ST15 (housing stock columns)   |
+-----+

cim_network schema:
network_scenarios (PK: grid_id UUID)
scenario_buses    (grid_id FK, bus_id)
scenario_lines    (grid_id FK, line_id)
network_buses     (PK: bus_id, geometry POINT)
network_lines     (PK: line_id, geometry LINESTRING)

cim_raster schema (height/z calculators use dtm, dsm):
dtm, dsm          PostGIS raster (DEM/DSM)
dtm_raster,       Alternative storage (RasterService)
dsm_raster
building_height_cache  Cached DSM-DTM heights per building
```

4.2 Outputs schema (TimescaleDB)

The `outputs` schema stores simulation time-series data as TimescaleDB hypertables, partitioned on `time_step` (BIGINT, discrete step index).

Table	Purpose	Partition key
<code>outputs.simulation_run</code>	Metadata per run (step_size_ms, simulation_start)	--
<code>outputs.building_frassinetto3</code>	Building thermal outputs (t_building, heating_load_target)	<code>time_step</code>
<code>outputs.battery</code>	Battery outputs (i, v, soc, p_net_batt)	<code>time_step</code>
<code>outputs.heating_frassinetto_hp2</code>	Heat pump outputs (en_el, cop, cr, ...)	<code>time_step</code>

Each hypertable row references `cim_vector.cim_wizard_building_properties` via a composite FK on (`scenario_id`, `building_id`, `lod`).

4.3 3DCityDB schema with Energy ADE and Utility Network ADE

The `citydb` schema is the standard 3DCityDB v4 schema for CityGML data. It is created by the official `create-db.sql` script at first container startup (see `cim-database/init-db/00_3dcitydb_setup.sh`). SRID is set to 4326 (WGS84).

Key tables used by CIM Wizard:

Table	Purpose	SQLAlchemy Model
<code>citydb.citymodel</code>	CityGML CityModel (one per scenario)	<code>CityModel</code>
<code>citydb.cityobject</code>	Root for all features (buildings, surfaces)	<code>CityObject</code>
<code>citydb.cityobject_member</code>	CityModel-to-CityObject link	<code>CityObjectMember</code>
<code>citydb.building</code>	CityGML Building (shares id with cityobject)	<code>CityBuilding</code>
<code>citydb.thematic_surface</code>	Wall/Roof/Ground surfaces (objectclass 33/34/35)	<code>ThematicSurface</code>
<code>citydb.surface_geometry</code>	Hierarchical geometry storage (PolygonZ)	<code>SurfaceGeometry</code>
<code>citydb.cityobject_genericattrib</code>	Key-value attributes (U-values, thermal zone data)	<code>CityObjectGenericAttrib</code>

The `citydb_pkg` schema contains PL/pgSQL functions for 3DCityDB maintenance.

SQLAlchemy models are in `app/models/citydb.py`. These models map to pre-existing tables; they are not created by `Base.metadata.create_all()`.

4.4 Willing schema (target ECDT)

Target schema for Energy Community Digital Twin, derived from `paper/data-list.md` and `paper/generate_data_list.xlsx.py`. Not yet implemented; serves as a roadmap.

Domain	Target tables / concepts	Data source
Geography	building_footprint, roof_geometry, thermal_zones, openings	cim_vector (extend), CityGML LoD2/3

Domain	Target tables / concepts	Data source
Grid	substation, feeder_topology, line_geometry, metering_points	cim_network (extend), CIM
Envelope	u_values (wall, roof, floor), glazing, infiltration	building_properties (extend), TABULA
Systems	hvac, dhw, lighting, appliances	New: cim_systems
Consumption	electricity, gas, heat (agg + disaggregated)	outputs (extend), smart meter, simulators
Occupant	occupancy_schedule, window_opening, thermostat_override	New: cim_occupant, occupant models
Generation	pv (extend), wind, biomass	cim_vector.pv (extend), registry
Storage & grid	battery, ev, import_export, tariffs	New: cim_storage, cim_grid_interaction
Weather	temperature, irradiance, wind	New: cim_weather (or external TMY)
Measurements	P, Q, V, meter_health, pseudo_measurements	New: cim_measurements (TimescaleDB)
Metadata	building_id, meter_id, provenance	Extend identifiers, add provenance

Synthesis note: Consumption and occupant behavior data can be **synthesized by simulators and models** (e.g. EnergyPlus, COESI, occupant behavior models) when real measurements are unavailable — see generation logic in [paper/generate_data_list.xlsx.py](#).

5. API Endpoint Reference

5.1 Core Endpoints

Method	Route	Purpose
GET	/api/v1/vector/projects	List all projects (with pagination)
GET	/api/v1/vector/pscenarios/{project_id}	Get scenarios for a project
GET	/api/v1/vector/project_scenario_details/{project_id}/{scenario_id}	Get details for a specific project scenario
GET	/api/v1/vector/get_buildings_geojson/{project_id}/{scenario_id}	Get buildings as GeoJSON FeatureCollection (includes building_properties, z_value, building_name, pv_data if available, grid info if available)
GET	/api/v1/vector/buildingproperties/{project_id}/{scenario_id}	Query building properties for a scenario (supports building_id, lod, pagination)
PUT	/api/v1/vector/projects/{project_id}/scenarios/{scenario_id}/buildingproperties	Flexible update/nullify building properties (single, batch, or bulk). See detailed docs below.
GET	/api/v1/vector/schema	Get normalization schema (all entities)
GET	/api/v1/vector/schema/{entity_name}	Get normalization schema for one entity
GET	/api/v1/building/lod12_geojson?project_id=...&scenario_id=...	Get LOD 1.2 surfaces as 3D GeoJSON FeatureCollection
POST	/api/v1/building/execute_building_analysis	Create project + run full 22-step pipeline (physical + demographic + 3D)
POST	/api/v1/complete/execute_complete_chain	<i>Deprecated</i> -- use building_analysis endpoint instead
DELETE	/api/v1/vector/delete	Flexible delete (building, scenario, or entire project)

5.2 Grid Endpoints

Method	Route	Purpose
POST	/api/v1/building/assign_grid	Assign a grid_id to a project scenario. Body: { "project_id": "...", "scenario_id": "...", "grid_id": "..." }
GET	/api/v1/building/grid/{project_id}/{scenario_id}	Get grid data (lines + buses) for a scenario that has a grid_id assigned
PUT	/api/v1/vector/projects/{project_id}/scenarios/{scenario_id}/clear-grid	Clear (set to NULL) the grid_id from a project scenario

5.3 PV Endpoints

Method	Route	Purpose
POST	/api/v1/building/assign_pv	Spatial-join PV polygons to buildings in a scenario. Updates <code>cim_vector.pv.building_id</code> and <code>cim_wizard_building.pv_ids</code> . Body: { "project_id": "...", "scenario_id": "..."} }
GET	/api/v1/building/pv_buildings/{project_id}/{scenario_id}	Get all PV polygons for buildings in the scenario as GeoJSON FeatureCollection, enriched with <code>building_height</code> and <code>z_value</code>

PV data is also included in the `get_buildings_geojson` response when a building has `pvs_ids` assigned.

5.4 3DCityDB / CityJSON Endpoints

There are two sets of CityJSON/CityDB endpoints:

A) CIM Wizard source (reads from `cim_vector` tables)

Method	Route	Purpose
POST	/api/v1/building/map_to_citydb	Map all buildings in a scenario to 3DCityDB tables (<code>citymodel</code> , <code>cityobject</code> , <code>building</code> , <code>thematic_surface</code> , <code>surface_geometry</code> , generic attributes). Idempotent. Body: { "project_id": "...", "scenario_id": "..."} }
GET	/api/v1/building/cityjson/{project_id}/{scenario_id}	Return CityJSON v1.1 built from CIM Wizard tables. Does NOT require <code>/map_to_citydb</code> .

B) 3DCityDB source (reads from `citydb` schema, requires `/map_to_citydb` first)

All endpoints below use `citymodel_id` which equals the `scenario_id` used during `POST /map_to_citydb`.

Method	Route	Purpose
GET	/api/v1/citydb/cityjson/{citymodel_id}	CityJSON v1.1 built from 3DCityDB tables (<code>citydb</code> schema)
GET	/api/v1/citydb/{citymodel_id}	CityModel metadata: envelope, name, member-count breakdown
GET	/api/v1/citydb/{citymodel_id}/cityobjects/{cityobject_id}	Any CityObject by numeric ID
GET	/api/v1/citydb/{citymodel_id}/buildings	List all buildings in the CityModel with generic attributes
GET	/api/v1/citydb/{citymodel_id}/buildings/{building_id}	Building detail: physical properties, energy system, thermal zone, surfaces
GET	/api/v1/citydb/{citymodel_id}/buildings/{building_id}/surfaces	List thematic surfaces (wall, roof, ground) with U-values
GET	/api/v1/citydb/{citymodel_id}/buildings/{building_id}/surfaces/{surface_id}	Surface detail with GeoJSON geometry and generic attributes

CityJSON mapping:

CIM Wizard concept	CityJSON / 3DCityDB representation
project-scenario	CityModel (gmlid = <code>scenario_id</code>)
building	Building CityObject (gmlid = <code>building_id</code>)
LOD 1.2 wall/roof/ground	Semantic surfaces in Solid geometry with TABULA U-values
thermal zone (1 per building)	+Energy-ThermalZone child (id = <code>building_id</code> + "-z1")
envelope_efficiency, <code>fmu_file</code>	ThermalZone attributes (<code>envelopeEfficiency</code> , <code>energySystemModel</code>)

TABULA U-values (W/m2K) applied per construction period:

Period	Years	Wall	Roof	Ground
TABULA_1	pre-1900	1.70	2.20	1.60
TABULA_2	1901-1920	1.60	2.00	1.50
TABULA_3	1921-1945	1.48	1.80	1.40
TABULA_4	1946-1960	1.30	1.60	1.20
TABULA_5	1961-1975	1.10	1.20	1.00
TABULA_6	1976-1990	0.80	0.80	0.80

Period	Years	Wall	Roof	Ground
TABULA_7	1991-2005+	0.50	0.40	0.50

5.5 Pipeline Endpoints (Generic)

Method	Route	Purpose
POST	/api/v1/pipeline/execute	Execute pipeline with feature list
POST	/api/v1/pipeline/execute_explicit	Execute with explicit feature.method plan
POST	/api/v1/pipeline/execute_predefined	Execute a named pipeline from configuration.json
POST	/api/v1/pipeline/calculate_feature	Calculate a single feature
GET	/api/v1/pipeline/configuration	Get full configuration.json
GET	/api/v1/pipeline/available_features	List available calculated features
GET	/api/v1/pipeline/predefined_pipelines	List predefined pipelines

5.6 DELETE Project Data

See below for details.

POST Create Project Request

POST /api/v1/building/execute_building_analysis
Content-Type: application/json

```
{
  "project_boundary": {
    "type": "Feature",
    "geometry": {
      "type": "Polygon",
      "coordinates": [[[lng1, lat1], [lng2, lat2], ..., [lng1, lat1]]]
    },
    "properties": {}
  },
  "project_name": "My Project",
  "scenario_name": "baseline",
  "save_to_db": true
}
```

Parameter	Type	Required	Default	Description
project_boundary	GeoJSON Feature	Yes	--	Polygon boundary in EPSG:4326
project_name	string	No	"Building_Analysis"	Human-readable project name
scenario_name	string or null	No	"baseline"	If null, defaults to "baseline" with scenario_id = project_id
save_to_db	boolean	No	true	Whether to persist results to database

DELETE Project Data

A single flexible endpoint that deletes a building, a scenario, or an entire project depending on which query parameters are provided. The `project_id` parameter is always required. Adding `scenario_id` narrows the scope to one scenario, and further adding `building_id` narrows it to a single building.

DELETE /api/v1/vector/delete?project_id=<id>[&scenario_id=<id>][&building_id=<id>]

Query Parameters

Parameter	Type	Required	Description
project_id	string	Yes	The UUID of the project
scenario_id	string	No	The UUID of the scenario within the project
building_id	string	No	The UUID of a specific building (requires <code>scenario_id</code>)

Behavior Matrix

project_id	scenario_id	building_id	Action
------------	-------------	-------------	--------

project_id	scenario_id	building_id	Action
provided	provided	provided	Delete one building -- removes its row from building_properties ; also removes the geometry from building if no other scenario references that building
provided	provided	--	Delete one scenario -- removes all building_properties for that scenario, orphaned building geometries, and the project_scenario record
provided	--	--	Delete entire project -- removes all building_properties , orphaned building geometries, and all project_scenario records for every scenario in the project

Example Requests (Postman / curl)

Delete a single building:

```
DELETE /api/v1/vector/delete?project_id=abc-123&scenario_id=def-456&building_id=ghi-789
```

Delete a scenario and all its buildings:

```
DELETE /api/v1/vector/delete?project_id=abc-123&scenario_id=def-456
```

Delete an entire project:

```
DELETE /api/v1/vector/delete?project_id=abc-123
```

Response

All three cases return a JSON object summarizing what was removed:

```
{
  "action": "delete_building | delete_scenario | delete_project",
  "project_id": "abc-123",
  "scenario_id": "def-456",
  "building_id": "ghi-789",
  "building_properties_deleted": 1,
  "buildings_deleted": 1,
  "project_scenarios_deleted": 0
}
```

For **delete_project**, the response also includes a **scenarios_affected** array listing all scenario IDs that were removed.

Safety Logic

- A **building** geometry row is only deleted when no other scenario still references that **building_id** in **building_properties**. This prevents data loss when multiple scenarios share the same physical building.
- If a **project_id** has no matching **project_scenario** records, the endpoint returns **404 Not Found**.
- On any unexpected error the transaction is rolled back and a **500 Internal Server Error** is returned with a description.

PUT Update/Nullify Building Properties

A single flexible endpoint that supports three modes for modifying building property fields.

```
PUT /api/v1/vector/projects/{project_id}/scenarios/{scenario_id}/buildingproperties
Content-Type: application/json
```

Editable fields

height, area, volume, number_of_floors, type, const_period_census, const_year, const_tabula, n_people, n_family, envelope_efficiency, fmu_file

Mode 1: Update a single building

Send a flat object with **building_id** and the fields to update. Setting a value to **null** clears it in the database.

```
{
  "building_id": "000d3106-3e9f-47b2-8ec6-fbe20edb4a8a",
```

```
"height": 14.2,
"n_people": null,
"envelope_efficiency": "high"
}
```

Response:

```
{
  "upserted": 1,
  "scenario_id": "def-456"
}
```

Mode 2: Update multiple buildings in one request

Wrap modifications in a `modifications` array. Each entry requires a `building_id`.

```
{
  "modifications": [
    { "building_id": "000d3106-...", "height": 14.2, "type": "SFH" },
    { "building_id": "11223344-...", "height": null, "n_people": 3 },
    { "building_id": "55667788-...", "envelope_efficiency": null }
  ]
}
```

Response:

```
{
  "upserted": 3,
  "scenario_id": "def-456"
}
```

Mode 3: Bulk update all buildings in the scenario

Omit `building_id` and provide a `fields` object. Every building in the scenario is updated with the given values.

```
{
  "fields": {
    "envelope_efficiency": "medium",
    "fmu_file": null
  }
}
```

Response:

```
{
  "action": "bulk_update",
  "project_id": "abc-123",
  "scenario_id": "def-456",
  "fields_updated": ["envelope_efficiency", "fmu_file"],
  "buildings_affected": 54
}
```

PUT Clear Grid ID

Remove the grid association from a project scenario.

```
PUT /api/v1/vector/projects/{project_id}/scenarios/{scenario_id}/clear-grid
```

No request body is needed. Response:

```
{
  "success": true,
  "project_id": "abc-123",
  "scenario_id": "def-456",
}
```

```
"grid_cleared": true
}
```

`grid_cleared` is `true` if a `grid_id` was present and removed, `false` if the scenario had no `grid_id`.

GET PV Buildings

Return all PV polygons for buildings in a project scenario as a GeoJSON FeatureCollection, enriched with building height and terrain elevation.

```
GET /api/v1/building/pv_buildings/{project_id}/{scenario_id}?lod=0
```

Response:

```
{
  "type": "FeatureCollection",
  "project_id": "53985b6a-...",
  "scenario_id": "53985b6a-...",
  "total_pv": 312,
  "features": [
    {
      "type": "Feature",
      "geometry": { "type": "MultiPolygon", "coordinates": [...] },
      "properties": {
        "pv_id": "8ba5569f-...",
        "building_id": "000d3106-...",
        "fid": 29116,
        "slope": 33.39,
        "area_reale": 8.08,
        "number": 4,
        "s": 6.9,
        "building_height": 12.5,
        "z_value": 287.3
      }
    }
  ]
}
```

3DCityDB / CityJSON Workflow

The CityDB integration follows a two-step workflow:

Step 1: Map CIM Wizard data to 3DCityDB

```
POST /api/v1/building/map_to_citydb
Content-Type: application/json

{ "project_id": "53985b6a-...", "scenario_id": "53985b6a-..." }
```

Response:

```
{
  "citymodel_id": 1,
  "scenario_id": "53985b6a-...",
  "mapped_buildings": 54,
  "total_buildings": 54
}
```

This creates a CityModel in the `citydb` schema with `gmId` = `scenario_id`. Each building becomes a CityObject with thematic surfaces, geometry, and generic attributes (TABULA U-values, thermal zone data).

Step 2: Query the CityModel via the hierarchical REST API

The `citymodel_id` in all GET endpoints below is the `scenario_id`.

Get the CityModel metadata:

```
GET /api/v1/citydb/53985b6a-...
```

```
{
  "id": 1,
  "gmlid": "53985b6a-...",
  "name": "CIM-Scenario-53985b6a",
  "members": { "total": 216, "Building": 54, "WallSurface": 108, "RoofSurface": 27, "GroundSurface": 27 }
}
```

List all buildings:

```
GET /api/v1/citydb/53985b6a-.../buildings
```

```
{
  "citymodel_id": "53985b6a-...",
  "total_buildings": 54,
  "buildings": [
    {
      "gmlid": "000d3106-...",
      "name": "BUI-0001",
      "measured_height": 12.5,
      "storeys_above_ground": 4,
      "envelope_efficiency": "medium",
      "thermal_zone_volume_m3": 3750.0,
      "surface_count": 6
    }
  ]
}
```

Get a single building with all detail:

```
GET /api/v1/citydb/53985b6a-.../buildings/000d3106-...
```

```
{
  "gmlid": "000d3106-...",
  "name": "BUI-0001",
  "building": {
    "measured_height": 12.5,
    "measured_height_unit": "m",
    "storeys_above_ground": 4
  },
  "energy_system": {
    "envelope_efficiency": "medium",
    "fmu_file": "frassinetto3"
  },
  "thermal_zone": {
    "volume_m3": 3750.0,
    "floor_area_m2": 300.0,
    "number_of_floors": 4
  },
  "surfaces": [
    { "gmlid": "1-wall-N", "surface_type": "WallSurface", "u_value_w_m2k": 1.30 },
    { "gmlid": "1-roof", "surface_type": "RoofSurface", "u_value_w_m2k": 1.60 },
    { "gmlid": "1-ground", "surface_type": "GroundSurface", "u_value_w_m2k": 1.20 }
  ]
}
```

Get a surface with geometry:

```
GET /api/v1/citydb/53985b6a-.../buildings/000d3106-.../surfaces/1-wall-N
```

```
{
  "gmlid": "1-wall-N",
  "surface_type": "WallSurface",
  "u_value_w_m2k": 1.30,
  "geometries": [
    {
      "gmlid": "poly-2",
      "geometry": { "type": "Polygon", "coordinates": [...] }
    }
  ]
}
```

```
    }
  ]
}
```

Get CityJSON v1.1 from the 3DCityDB schema:

```
GET /api/v1/citydb/cityjson/53985b6a-...
```

This returns a full CityJSON v1.1 document built from the `citydb` tables, including Building CityObjects with LOD 1.2 Solid geometry, semantic surfaces with TABULA U-values, and +Energy-ThermalZone child objects.

Alternatively, get CityJSON directly from CIM Wizard tables (no `map_to_citydb` step needed):

```
GET /api/v1/building/cityjson/53985b6a-.../53985b6a-...
```

Both endpoints return the same CityJSON v1.1 structure. The `citydb` variant reads from the mapped 3DCityDB tables; the `building` variant reads from CIM Wizard tables directly.

6. Backend Field Normalizer

Problem

Different clients send the same data using different field names:

- Frontend (React): `projectName`, `scenarioName`, `n_families`
- Postman users: `project_name`, `Project_Name`
- GIS tools / Italian datasets: `altezzaa_vo`, `superficie`, `epoca_cost`

Without centralized normalization, every client must independently figure out the correct field names, leading to drift and silent data loss.

Solution: Config-Driven Normalizer

```
app/core/
+-- normalization_config.json  <-- JSON config: canonical names + all known aliases
+-- normalizer.py              <-- Python module: normalize, validate, schema export
```

On input (POST/PUT): The normalizer maps any known alias to the canonical name before processing. A client can send `projectName` or `proj_name` or `ProjectName` -- they all resolve to `project_name`.

On output (GET): The backend always returns canonical names (matching DB columns). Clients can query the `/schema` endpoint to discover what those names are.

Schema Discovery Endpoint

Any client can fetch the full normalization schema:

```
GET /api/v1/vector/schema          -- Full schema (all entities)
GET /api/v1/vector/schema/building_properties  -- Single entity schema
```

Entities Covered

Entity	DB Table	Fields	Description
<code>project_scenario</code>	<code>cim_vector.cim_wizard_project_scenario</code>	11	Project/scenario metadata + boundaries
<code>building</code>	<code>cim_vector.cim_wizard_building</code>	8	Building geometry records
<code>building_properties</code>	<code>cim_vector.cim_wizard_building_properties</code>	15	Physical + demographic properties per scenario

Adding New Aliases

Edit `normalization_config.json`:

```
"height": {
  "aliases": ["Height", "altezzaa_vo", "building_height", "h",
             "NEW_ALIAS_HERE"]
}
```

No Python code changes needed. Restart the server or call `reload_config()`.

7. Issues and Recommendations

Critical Issues

#	Issue	Location	Impact	Status
1	<code>complete_chain_route.py</code> is deprecated	<code>complete_chain_route.py</code>	Non-UUID IDs, no DB saving, wrong method names	Use <code>building_analysis_route.py</code> which now runs all 22 steps

Minor Issues

#	Issue	Location	Impact
2	No explicit foreign keys in most CIM Wizard tables	cim-database schema	No referential integrity enforcement (by design -- maintained by app logic)
3	Frontend normalizer field mismatches	<code>normalizers.ts</code>	Frontend should call <code>GET /schema</code> to get canonical names
4	Energy ADE / Utility Network ADE may not install cleanly	<code>cim-database/init-db/00_3dcitydb_setup.sh</code>	The gioagu ADE scripts are designed for 3DCityDB v3/v4 and may produce warnings on the edge image. Core 3DCityDB schema works correctly.

8. Running the System

Prerequisites

- Docker and Docker Compose
- Python 3.10+ with conda or pip
- Node.js 18+

Docker Quick Start (DB + Backend)

```
chmod +x run-docker.sh
./run-docker.sh up      # Start database and backend
./run-docker.sh down    # Stop both
```

1. Start the Database

```
cd cim-database
docker compose -f docker-compose.cimdb.yml up -d
# Verify: docker logs cim-integrateddb --tail 20
```

2. Start the Backend

```
cd cim_wizard_integrated_2026
conda env create -f environment.yml
conda activate cim_wizard
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
# API docs at: http://localhost:8000/docs
```

3. Start the Frontend

```
cd coesi-frontend-main_2026
npm install
npm run dev
# Frontend at: http://localhost:5173
```

Environment Variables

Frontend (`.env` in `coesi-frontend-main_2026/`):

```
VITE_CIM_WIZARD_BASE=http://localhost:8000
```

Backend (environment or `.env`):

```
DATABASE_URL=postgresql://cim_wizard_user:cim_wizard_password@localhost:15432/cim_wizard_integrated
```

Connection Summary

Service	Host	Port	Credentials
PostgreSQL/PostGIS + TimescaleDB + 3DCityDB	localhost	15432	cim_wizard_user / cim_wizard_password
FastAPI Backend	localhost	8000	--
React Frontend	localhost	5173	--

9. Deployment to Server

The production server is at 130.192.238.11. The deployment script deploy-to-server.sh uses rsync + sshpass to push local code.

Full redeployment (code + fresh DB)

```
# 1. Deploy code
./deploy-to-server.sh

# 2. On the server
ssh eclabuser@130.192.238.11
cd ~/cim
./run-docker.sh down

# 3. Rebuild DB (destroys data -- only for fresh setup)
cd cim-database
docker compose -f docker-compose.cimdb.yml build --no-cache
docker compose -f docker-compose.cimdb.yml up -d

# 4. Restart backend
cd ~/cim
./run-docker.sh backend-up
```

Incremental deployment (keep existing data)

When only schema changes or code updates are needed and existing data must be preserved:

```
# 1. Deploy code
./deploy-to-server.sh

# 2. On the server: restart backend only (no DB rebuild)
ssh eclabuser@130.192.238.11
cd ~/cim
./run-docker.sh backend-down
./run-docker.sh backend-up

# 3. Apply SQL migrations if needed (additive, IF NOT EXISTS)
docker exec -i cim-integrateddb psql -U cim_wizard_user -d cim_wizard_integrated \
< cim-database/migrations/add_outputs_schema.sql

docker exec -i cim-integrateddb psql -U cim_wizard_user -d cim_wizard_integrated \
< cim-database/migrations/add_grid_id_building_cols_pv_table.sql

# 4. Install 3DCityDB on existing database (if not already done)
docker exec -i cim-integrateddb bash -c '
psql -v ON_ERROR_STOP=1 -U cim_wizard_user -d cim_wizard_integrated \
-f /3dcitydb/create-db.sql \
-v srid=4326 \
-v srs_name="urn:ogc:def:crs:EPSG::4326" \
-v changelog=no
'
```

Loading PV data

To load PV GeoJSON data into cim_vector.pv:

```
conda activate webgis
cd pv/scripts
python load_pv_geojson.py          # local DB
python load_pv_geojson.py --server # server DB (130.192.238.11)
```

